

Computer Science Internal Assessment Criterion C - Development

Word Count : 1093

Techniques, frameworks and, libraries used

Techniques used	
1. Saving and retrieving data from the database 2. Asynchronous functions with promises and callbacks (fetch, db.query, bcrypt.hash/compare) 3. Authentication and validation using bcrypt and express-session 4. Sending email notifications using Nodemailer 5. Chart manipulation using chartJS 6. Role-Based Access Control (RBAC) through session objects 7. Generating and exporting PDF reports using jsPDF and canvas 8. Secure storage of environment variables using dotenv 9. Random verification (OTP) codes for password reset using crypto	
Front-end library/frameworks	Back-end library/frameworks
HTML - Chart JS - Fetch API (HTTP requests) → for sending/receiving JSON data between frontend and backend.	Node JS - Nodemailer - Express.js
CSS	MySQL - Managed using MySQL workbench
JavaScript	Bcrypt - hashing & verifying passwords
	Utility Libraries - jsPDF + canvas → PDF generation - path → file path resolution - dotenv → environment variable storage - crypto → random verification codes

Sending and receiving data between backend and frontend (CRUD operations)

Backend

The ability of the application to send and receive data between the backend and frontend relies on **CRUD operations** implemented through the **Express.js framework** in conjunction with

MySQL.¹ Express handles HTTP requests for interacting with the database by providing built-in methods (e.g., `app.get`, `app.post`), which map to various CRUD operations.²

```
// MySQL Connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: process.env.DB_PASSWORD,
  database: 'LoginDB' // The database password is stored in a local file called .env as it shouldn't be shared.
});
//The backend communicates with the relational database using the MySQL dependency, which establishes access to the library.
```

Code 1) MySQL Connection in `server.js`, enables backend communication with the database.

The following shows a route that requests to GET a certain resource. **This role-based access control prevents unauthorised access to sensitive data.**

```
app.get('/get-users', (req, res) => {
  if (req.session.user && req.session.user.role === 'admin') {
    //this conditional statement is used to check if a user is logged in and whether their role is admin.
    const query = 'SELECT id, name, department, role FROM users';
    db.query(query, (err, results) => {
      if (err) return res.status(500).send('Error fetching users.');
```

Code 2) `app.get('/get-users')`, which checks if the user is admin and displays data accordingly.

¹ “The Pros and Cons of Node.JS in Web Development.” GeeksforGeeks, 5 Dec. 2022, www.geeksforgeeks.org/node-js/the-pros-and-cons-of-node-js-in-web-development/. Accessed 8 May 2025.

² Pandey, Deepak. “CRUD in Express - Scaler Topics.” *Scaler Topics*, 22 May 2023, www.scaler.com/topics/expressjs-tutorial/crud-in-express-js/. Accessed 8 May. 2025.

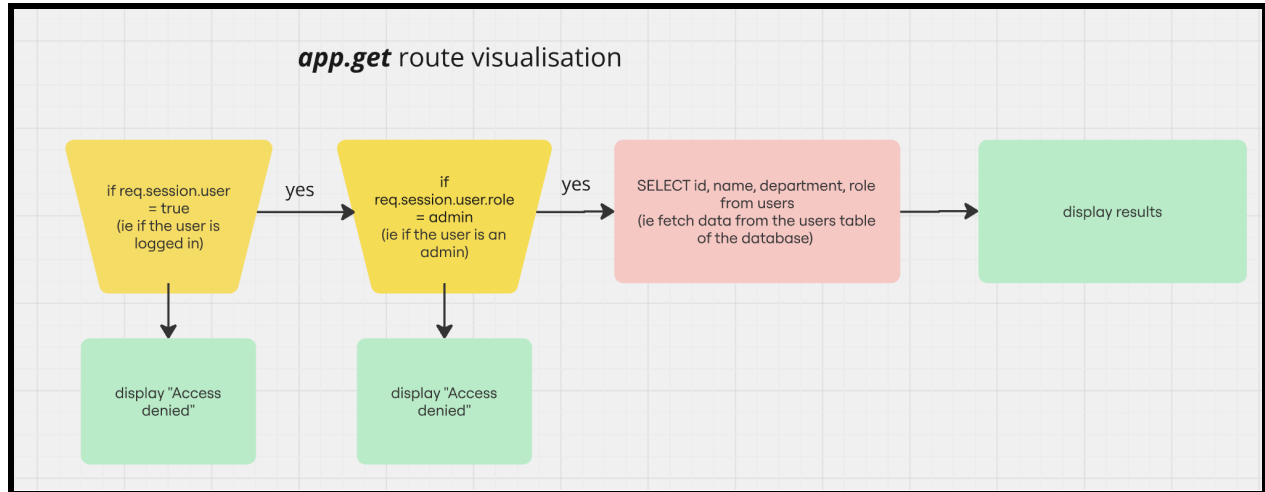


Diagram 1. a) Visual representation of the app.get() route in Code 2)

Most data processing is done on the backend through grouped aggregation, where rows of data in SQL are grouped together based on their columns.³

³ GeeksforGeeks. “How to Group and Aggregate Data Using SQL?” *GeeksforGeeks*, 9 Oct. 2021, www.geeksforgeeks.org/sql/how-to-group-and-aggregate-data-using-sql/. Accessed 8 May. 2025.

This code is an example of the same.

```
app.get('/admin-dashboard-data', (req, res) => {
  /*
  grouped aggregation - all expenditures are grouped by their department_id instead of
  sending separate requests for each department's expenditures.

  benefits - This design reduces the total number of HTTP calls the frontend needs to make, improving performance and responsiveness
  at runtime.
  drawbacks - it also means that the initial payload is larger, which may increase the initial page load time - sacrificing latency
  for bandwidth.
  */

  if (req.session.user && req.session.user.role === 'admin') {
    const query = `
      SELECT d.department_id, d.department_name, d.requested_budget, d.admin_comments,
             d.budget_status, u.username AS department_head,
             COALESCE(SUM(e.expenditure_amount), 0) AS actual_spent
      FROM departments d
      LEFT JOIN users u ON d.department_head_id = u.id
      LEFT JOIN expenditures e ON d.department_id = e.department_id
      GROUP BY d.department_id, d.department_name, d.requested_budget,
             d.admin_comments, d.budget_status, u.username
    `;
    /*department_id is the foreign key in the expenditures table, and is used to sort expenditures by department.
    user_id is the foreign id in the department table, and is used to show which user is the department head for the
    expenditure-department. hence all relevant data can be displayed.
    */
    db.query(query, (err, results) => {
      if (err) return res.status(500).send('Error fetching dashboard data.');
```

Code 3) app.get('/admin-dashboard-data), which groups expenditures by department before displaying them on the admin dashboard page.

After executing queries, the backend structures the response as a **single JSON object**. This object is sent back to the frontend in one HTTP response :

```
res.json({ //the json object contains both departmental data and aggregated expenditures
  departments: departmentsResults,
  expenditures: expendituresResults
});
/*
For each of the expenditure records that are retrieved, the details (department_ID, department_name, etc.)
are requested from the database.
*/
});
```

Code 4) JSON object, which contains the values of both department data and expenditures from the database.

Update operations allow administrators to make **real-time modifications** to existing records in the database.

The endpoint shown below — `/update-comment` — (**O(1)** with respect to data retrieval complexity) is a POST request, receiving JSON payloads from the frontend. Express parses these requests via middleware (`body-parser`) before passing them to the MySQL query layer.⁴

```
// === Update Comment === //
app.post('/update-comment', (req, res) => {
  const { department_id, comment } = req.body;
  const query = 'UPDATE departments SET admin_comments = ? WHERE department_id = ?';
  db.query(query, [comment, department_id], (err) => {
    if (err) return res.status(500).send('Error updating comment.');
```

Code 5) `app.post('/update-comment')`, which receives the updated values through JSON payloads, parses them and then passes them to the database to get stored.

The delete operation demonstrates how the backend enables **controlled removal of data** from the system. **From an algorithmic standpoint, deletion is O(1) for indexed primary keys (id), ensuring scalability even as the user base grows.** This makes it efficient for frequent administrative tasks, such as revoking access for employees who leave the organization.

```
app.post('/delete-user', (req, res) => { //this route handles administrator-initiated requests to remove a user from the users table.
  const { id } = req.body;
  const query = 'DELETE FROM users WHERE id = ?';
  /* this is hard deletion - the user is removed permanently, meaning historical data associated with that user (e.g., budget submissions)
  may lose referential context.
  an alternate solution is soft deletion - (e.g., is_active = false), which preserves referential integrity at the cost of
  increased storage
  for this system - hard deletion was appropriate, as administrators explicitly requested the ability to purge inactive
  accounts from the system.

  */
  db.query(query, [id], (err) => {
    if (err) return res.status(500).send('Error deleting user.');
```

Code 6) `app.post('/delete-user')`, which removes a user and their details from the database.

User Differentiation (Role-Based Access Control)

Backend

The ability of the application to differentiate users and restrict access to features is achieved through **role-based access control (RBAC)**, implemented using **Express.js sessions** in conjunction with MySQL.

⁴ “Express Body-Parser Middleware.” *Expressjs.com*, expressjs.com/en/resources/middleware/body-parser.html. Accessed 8 May 2025.

This ensures that every subsequent request contains information about the user's **identity and privileges**, enabling the server to enforce access control rules before serving HTML pages or processing database queries.⁵

```
// Hence, once a user logs in & their password is verified, their role is retrieved from the database and stored in the
// req.session.user object.
// This means that for every future request, the user's role can be checked.
req.session.user = {
  id: user.id, //Uniquely identifies the user
  username: user.username, //Displays the name of the user
  role: user.role //Used for access control - checks if user is admin, department head or general employee
}
/*Store user info in session so it persists across requests
// Store user info in session so it persists across requests */
};

if (user.role === 'admin') return res.redirect('/admin');
//redirects user to the admin page if their role is stored in the session object as 'admin'
// same logic is followed for other roles.

else if (user.role === 'General Employee') return res.redirect('/employee-dashboard');
else if (user.role === 'Department Head') return res.redirect('/department-head');
/* once the req.session.user object stores the user's details from the database, the corresponding role is used
to direct them to the appropriate dashboard.
*/
else return res.send('Login successful. But no dashboard defined for your role.');
```

```
/* this means that although the user has a valid username and password, their role is not defined
in the database. Hence they will not be redirected to a dashboard.
*/
});
});
```

Code 7) The req.session.user object stores the user's role in the session, hence the backend avoids repeatedly querying the database for authorisation on every action, increasing efficiency and directing users to the correct dashboard..

⁵ Frontegg. "What Is Role-Based Access Control (RBAC)? A Complete Guide." Frontegg, 15

Mar. 2022, frontegg.com/guides/rbac. Accessed 8 May 2025.

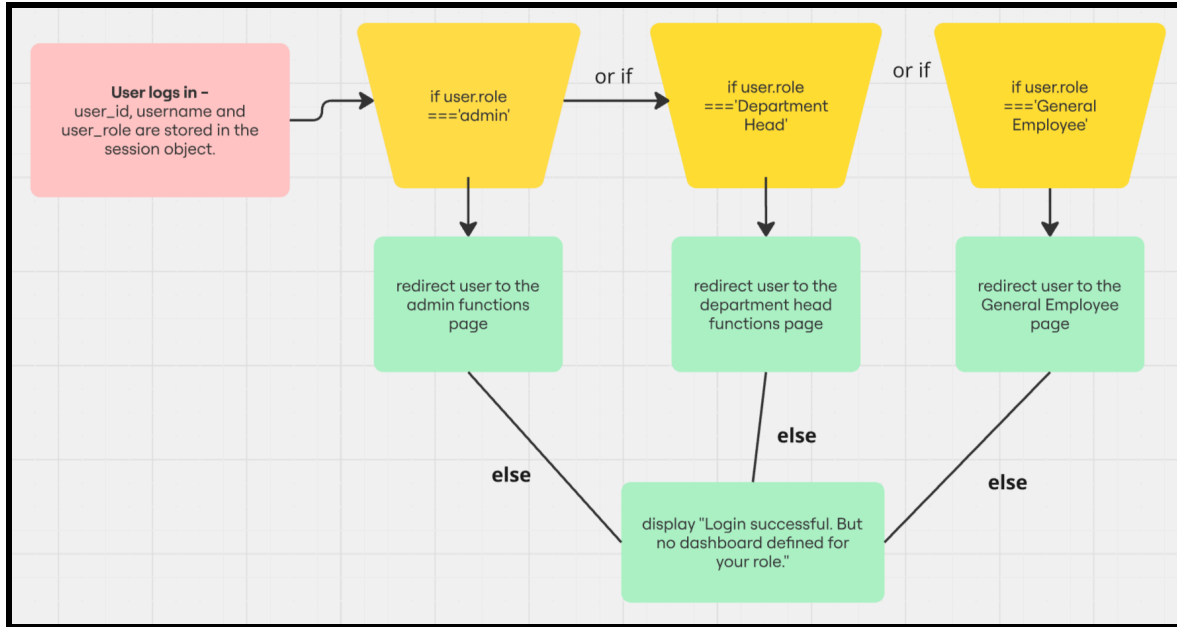


Diagram 2.a) How the session object authenticates users based on their role and directs them to the appropriate webpage.

The following route demonstrates how RBAC is enforced for **admin users**.

```

// Admin Routes
app.get('/admin', (req, res) => { //Checks if a user is logged in and has the role 'admin'
/*
When a request is made to /admin, the server checks if the session exists and if the role is equal to admin.
Only then is the requested file served; otherwise, access is denied.
*/
  if (req.session.user && req.session.user.role === 'admin') {
    //check if the user is stored as an admin in the database
    res.sendFile(path.join(__dirname, 'public', 'admin.html'));
    //when the if condition passes, the admin dashboard webpage is sent to the user's browser
  } else {
    res.send('Access denied. Admins only.');
    // If condition fails, unauthorised access is denied immediately
  }
  // This route protects sensitive admin dashboard from being accessed by non-admin users
});

```

Code 8) `app.get('/admin')`, which uses the user details stored in the session object to verify that the user is an admin.

This design ensures that administrative tasks such as managing users or reviewing budget requests remain restricted to a single, privileged role.

While all admins can see the same data, Department Heads have role-specific routes that only allow access to data of their own department.

This is implemented in Express.js using conditional checks on the user's session.⁶ When a Department Head requests the endpoint `/api/department-head/employees`, the backend first validates the user role through the session object (`req.session.user.role`). Once the role is confirmed, the system uses the helper function `getDeptHeadDepartmentId(req, callback)` to retrieve the `department_id` associated with the logged-in Department Head from the database.. The results are returned as a JSON object to the frontend, which then renders them in the Department Head's dashboard.

```
app.get('/api/department-head/employees', (req, res) => {
  if (!req.session.user || req.session.user.role !== 'Department Head') { //first checks the user's role
    return res.status(403).send('Access denied. Department Heads only.');
```

// If the logged-in user is not a Department Head, the server returns an HTTP 403 error, denying access to the resource.

```
  }
  getDeptHeadDepartmentId(req, (err, departmentId) => { //Retrieve the department_id for this head
    if (err) return res.status(500).send('Error resolving department.');
```

/*

Once resolved, the department_id is then passed into a parameterized SQL query fetching only the employees belonging to the same department as the logged-in Department Head

*/

```
    const query = `
      SELECT u.id, u.username, u.role, u.email, d.department_name
      FROM users u
      JOIN departments d ON u.department_id = d.department_id
      WHERE u.department_id = ?
      ORDER BY u.username ASC
    `; // ✅ Query employees only within this department
    /*Once obtained from the database, the results are returned as a JSON object to the frontend, which then renders them in the Department Head's dashboard.*/

    // Ensures Department Heads cannot query employees outside their department
    // Department ID is derived from the session, not from user input → prevents tampering
    // Department ID is derived from the session, not from user input → prevents tampering

    db.query(query, [departmentId], (qErr, results) => {
      if (qErr) return res.status(500).send('Error fetching employees.');
```

res.json(results || []);

```
    });
  });
});
```

Code 9) `app.get('/api/department-head/employees)`, which verifies the user's role and department using the details stored in the session object, and displays data accordingly.

⁶ "Express Routing." *Expressjs.com*, 2017, expressjs.com/en/guide/routing.html. Accessed 8 May 2025.

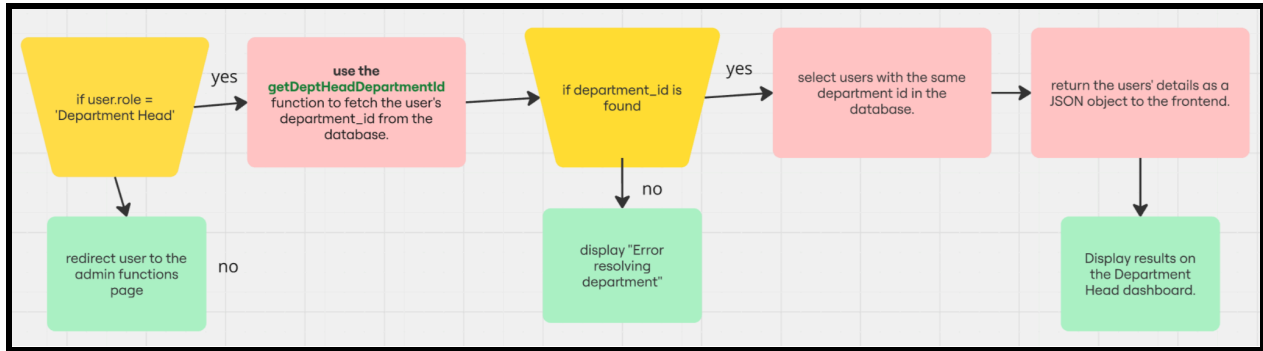


Diagram 2.b) How role-based access is used to show employee details depending on the `department_id` of the department head.

Frontend

On the frontend, user differentiation is implemented by dynamically rendering interface elements based on the role-specific data retrieved from the backend. Since RBAC is enforced server-side, the frontend focuses on fetching filtered datasets and rendering role-specific dashboards, exemplifying sound full-stack design where presentation and interaction reside client-side while access control and data integrity are maintained on the backend.

An example of this is given below.

```

function renderEmployees(data) {
  /* when a Department-Head accesses their dashboard, the client executes
  | an asynchronous fetch request to the endpoint /api/department-head/employees. */
  const table = document.getElementById('employee-details-table');
  // This ensures Department Heads only see employees belonging to their department,
  // since the backend already filtered 'data' before sending it. (see diagram 2.b)
  table.innerHTML = '';
  data.forEach(user => {
    // Iterates through the restricted dataset returned from the backend.
    // Each 'user' object corresponds to an employee in the Department Head's department only.
    const tr = document.createElement('tr');
    tr.innerHTML = `
      <td>${user.username}</td>
      <td>${user.department_name}</td>
      <td>${user.role}</td>
      <td>${user.email}</td>
      <td><button onclick="editUser(${user.id}, '${user.username}', '${user.role}', '${user.email}')">Edit</button></td>
      <td><button onclick="deleteUser(${user.id})">Remove</button></td>
    `;
  });
  //The backend hence responds with a JSON object containing only the employees belonging to the logged-in Department Head's department
  // The JavaScript in the HTML then iterates over this array and dynamically generates HTML table rows with each employee's details:
  //dashboard is populated exclusively with employees from the Department Head's department
  // this enforces role-based differentiation in the frontend view.

  /* From an efficiency perspective, this approach offloads all filtering and authorisation logic to the server,
  so the client only processes and displays pre-validated data. */
}

```

Code 10) function `renderEmployees(data)`, which uses the JSON object and the `api/department-head/employees` route to display relevant data

Bycrypt

One of the critical aspects of the application's security is the storage and verification of user passwords. Rather than storing plaintext passwords in the database, the backend employs the **bcrypt** hashing library to transform them into cryptographic hashes before insertion.⁷ This ensures that even if the database were compromised, the actual user passwords would not be retrievable in human-readable form.

This design guarantees that the database only ever stores the hashed output, not the original password -

```
const hashedPassword = await bcrypt.hash(newPassword, saltRounds);
//when registering a new administrator, the password entered by the user is first passed through this function

/* 'saltRounds' defines the cost factor (number of iterations) → more secure but slower
   - The saltRounds parameter controls the computational complexity of the hashing process by introducing unique salts
   and performing multiple iterations of hashing.
   - This makes brute-force or rainbow-table attacks computationally expensive. */
db.query(
  'UPDATE users SET password = ? WHERE username = ?',
  // SQL UPDATE query to replace the old password with the new hashed password
  //The resulting hash is inserted into the users table, replacing the plaintext value

  [hashedPassword, req.session.resetUsername], // Use parameterized values to prevent SQL injection
  (err) => {
    if (err) return res.send('Error updating password.');
```

Code 11) hashing the new password once it is reset by a user and updating it in the users table of the database.

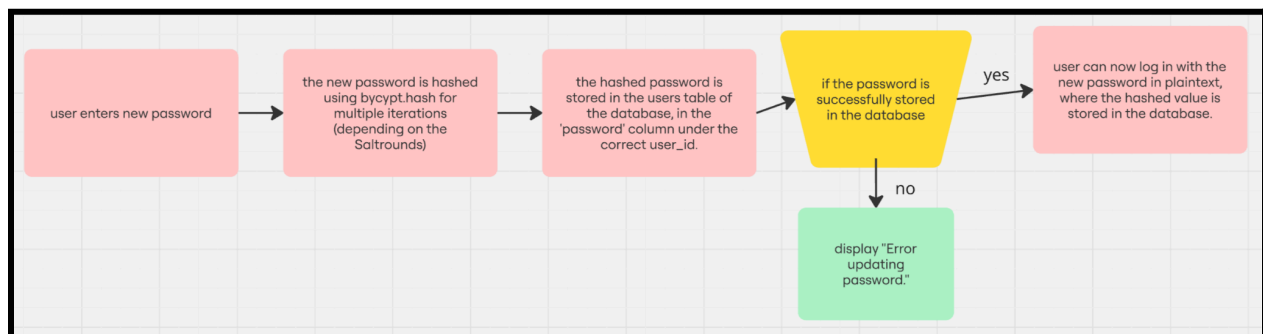


Diagram 3.a) How passwords are hashed and stored after a user resets them.

⁷ “What Is Bcrypt?” *JumpCloud*, 22 Apr. 2025, jumpcloud.com/it-index/what-is-bcrypt. Accessed 8 May 2025.

When a user attempts to log in, bcrypt is again employed—this time to compare the supplied password with the stored hash :

```
/* - bcrypt.compare() takes the plaintext password entered by the user and compares it
   | with the hashed password stored in the database.
   |
   | - when a user logs in, instead of re-hashing and directly matching strings, bcrypt internally handles
   |   salting and hashing, ensuring that the comparison is accurate and resistant to tampering
   |
   | - for this application, cost factor (saltRounds = 10) was selected as a balance between efficiency and security,
   |   following recommended industry practices.
   */

const isMatch = await bcrypt.compare(password, user.password);

/* If the result of the comparison is false, the system immediately denies access.
   This prevents users with incorrect credentials from authenticating
   */
if (!isMatch) return res.send('Error: Incorrect password.');
```

/* If the comparison fails - access is immediately denied.
If it succeeds - the user's credentials are authenticated, and their role (admin, Department Head, or General Employee)
is stored in the session object for subsequent access control.
*/

Code 12) comparing the plaintext password entered by the user to its hashed equivalent in the database

The trade-off of this implementation is computational overhead: bcrypt has **time complexity proportional to $O(2^{\text{saltRounds}})$** . This deliberately slows down hashing operations, which increases security against brute-force attacks but slightly impacts performance during login or registration.

Session Management

The backend uses the `express-session` dependency to manage state across multiple HTTP requests. Since HTTP is a stateless protocol, session management is required to preserve user identity once they have authenticated, instead of forcing re-login on every request.⁸

⁸ Authgear. “What Is Session Management: Threats and Best Practices - Authgear.” *Www.authgear.com*, 20 Aug. 2024, www.authgear.com/post/session-management. Accessed 8 May 2025.

At the start of the application, the middleware is configured as the following:

```
app.use(session({
  secret: 'your-secret-key',
  //A cryptographic string used to sign the session ID cookie
  // ensures integrity so clients cannot tamper with their own session ID.

  resave: false,
  // Prevents session data from being re-saved to the store if nothing has changed →
  // improves performance and reduces unnecessary writes.

  saveUninitialized: true
  //Forces a new but empty session to be stored even if no data is set →
  // ensures that a unique session ID is issued for every visitor, which is later be populated on login.
}));
```

Code 13) app.use(), which implements session management to ens

ure data integrity

Once a user logs in successfully, their credentials are verified with `bcrypt.compare()` and a session object is created. This object stores the authenticated user's information in server memory, which persists across subsequent requests. The database provides the user's `id` and `role`, but these are cached in the session to avoid repeated lookups on every route.

Final Word Count : 1093

Bibliography

Pandey, Deepak. "CRUD in Express - Scaler Topics." *Scaler Topics*, 22 May 2023, www.scaler.com/topics/expressjs-tutorial/crud-in-express-js/. Accessed 8 May. 2025.

GeeksforGeeks. "How to Group and Aggregate Data Using SQL?" *GeeksforGeeks*, 9 Oct. 2021, www.geeksforgeeks.org/sql/how-to-group-and-aggregate-data-using-sql/. Accessed 8 May. 2025.

“Express Body-Parser Middleware.” *Expressjs.com*,
expressjs.com/en/resources/middleware/body-parser.html. Accessed 8 May. 2025

Frontegg. “What Is Role-Based Access Control (RBAC)? A Complete Guide.” Frontegg, 15 Mar. 2022,
frontegg.com/guides/rbac. Accessed 8 May 2025.

“Express Routing.” *Expressjs.com*, 2017, expressjs.com/en/guide/routing.html.
Accessed 8 May 2025.

“The Pros and Cons of Node.JS in Web Development.” GeeksforGeeks, 5 Dec. 2022,
www.geeksforgeeks.org/node-js/the-pros-and-cons-of-node-js-in-web-development/. Accessed 8 May 2025.

“What Is Bcrypt?” *JumpCloud*, 22 Apr. 2025, jumpcloud.com/it-index/what-is-bcrypt.
Accessed 8 May 2025.

Authgear. “What Is Session Management: Threats and Best Practices - Authgear.”
Www.authgear.com, 20 Aug. 2024,
www.authgear.com/post/session-management.
Accessed 8 May 2025.

